CAHIER DES CHARGES — FRONTEND

Projet JobLink | Spécification détaillée de l'application React

Projet	JobLink — Plateforme de recrutement
Périmètre	Application frontend uniquement
Technologies	React.js · JavaScript · Tailwind CSS · React Router
Communication	API REST sécurisée fournie par le backend
Version	1.0 — Novembre 2026

Sommaire

• 1. Présentation et périmètre

• 2. Utilisateurs et droits d'interface

• 3. Technologies et conventions

• 4. Architecture et organisation des fichiers

• 5. Pages et exigences fonctionnelles

• 6. Navigation et protection des routes

• 7. Design UI/UX et responsive

• 8. Formulaires et validations frontend

• 9. Communication avec l'API

• 10. Sécurité côté frontend

• 11. Performance, qualité et tests

• 12. Critères de validation et livrables

• 13. Ordre conseillé de réalisation

1. Présentation et périmètre

Le présent document décrit exclusivement le **frontend** de JobLink, une plateforme web de recrutement permettant aux candidats de rechercher des offres et de suivre leurs candidatures, et aux recruteurs de publier des offres et de gérer les candidatures.

Le frontend est une application monopage (SPA) construite avec **React.js, JavaScript et Tailwind CSS**. Il communique avec le backend via une API REST sécurisée. Le frontend ne se connecte jamais directement à PostgreSQL.

Objectifs

- Fournir une interface claire, responsive et accessible sur ordinateur, tablette et mobile.
- Adapter les écrans et les actions au rôle de l'utilisateur connecté.
- Réduire les étapes nécessaires pour publier une offre ou postuler.
- Afficher les erreurs, confirmations, états de chargement et états vides de manière compréhensible.

Hors périmètre

- Règles métier exécutées exclusivement par le serveur.
- Gestion directe de la base PostgreSQL.
- Signature électronique ou paiement en ligne dans la première version.

2. Utilisateurs et droits d'interface

Profil	Accès frontend attendu
Visiteur	Accueil, liste et détail public des offres, inscription, connexion.
Candidat	Profil, CV, compétences, recherche, favoris, candidatures, entretiens, notifications.
Recruteur	Espace recruteur, gestion des entreprises autorisées, offres, candidatures et entretiens.
Administrateur	Tableau de bord, utilisateurs, entreprises, offres signalées et modération.

Le masquage d'un bouton dans React améliore l'expérience mais ne constitue pas une sécurité. Le backend doit toujours vérifier les permissions.

3. Technologies et conventions

- React.js avec composants fonctionnels et Hooks.
- JavaScript uniquement : aucun fichier TypeScript, aucun type TS.
- Tailwind CSS pour le système de styles et le responsive design.
- React Router pour les routes et layouts.
- Axios ou Fetch API pour les appels HTTP ; une seule solution sera retenue dans le projet.
- React Context ou une solution légère pour l'authentification et les états globaux.
- Bibliothèque de formulaires et validation côté client, avec règles alignées sur le backend.

Conventions

- Un composant par responsabilité ; composants réutilisables dans components/.

• Pages dans pages/ ; appels API dans services/ ; contexte dans context/ ; hooks personnalisés dans hooks/.

• Noms de composants en PascalCase et fonctions/variables en camelCase.

• Les secrets et clés privées ne sont jamais stockés dans le frontend.

4. Architecture et organisation des fichiers

```
src/
assets/
components/
common/ (Button, Input, Modal, Loader, Alert, Pagination)
layout/ (Navbar, Sidebar, Footer, DashboardLayout)
jobs/ (JobCard, JobFilters, JobForm)
applications/ (ApplicationCard, StatusBadge)
pages/
public/ (Home, Jobs, JobDetails, Login, Register)
candidate/ (Dashboard, Profile, Resume, Favorites, Applications)
recruiter/ (Dashboard, Company, Offers, OfferForm, Applicants)
admin/ (Dashboard, Users, Companies, Moderation)
routes/ (AppRouter, ProtectedRoute, RoleRoute)
services/ (apiClient, authService, jobService, applicationService)
context/ (AuthContext, NotificationContext)
hooks/
utils/ (validators, formatters, constants)
App.js
index.js
```

Cette organisation est indicative. Elle peut évoluer sans mélanger les pages, les composants visuels et la communication avec l'API.

5. Pages et exigences fonctionnelles

5.1 Pages publiques

Page	Contenu et actions
Accueil /	Présentation, recherche rapide, catégories, offres récentes, boutons Se connecter / S'inscrire.
Offres /jobs	Liste paginée, recherche par mot-clé, lieu, type de contrat, catégorie et filtres.
Détail /jobs/:id	Description, entreprise, localisation, compétences, date limite ; bouton Postuler ou Se connecter.
Connexion /login	Email, mot de passe, affichage/masquage du mot de passe, erreurs et redirection selon rôle.
Inscription /register	Choix candidat/recruteur selon règles du produit ; champs obligatoires, validation et confirmation.

5.2 Espace candidat

Page	Contenu et actions
Dashboard	Résumé des candidatures par statut, offres récentes recommandées et raccourcis.
Profil	Informations personnelles, photo facultative, localisation, résumé et coordonnées.
CV /resume	Ajouter, remplacer, télécharger ou supprimer un CV ; formats et taille autorisés affichés.

Page	Contenu et actions
Compétences	Ajouter/supprimer des compétences et niveaux si prévus par l'API.
Favoris	Lister et retirer les offres sauvegardées.
Candidatures	Historique, date, offre, statut et détail du suivi.
Entretiens	Date, heure, format, lieu/lien et consignes.
Notifications	Liste, lecture/non-lecture et accès à l'élément concerné.

5.3 Espace recruteur

Page	Contenu et actions
Dashboard	Nombre d'offres actives, candidatures reçues, entretiens à venir.
Entreprise	Informations, logo et membres autorisés selon les permissions.
Mes offres	Liste, recherche, filtres, création, modification, publication, clôture.
Créer/modifier une offre	Titre, description, missions, profil, compétences, lieu, contrat, salaire facultatif et date limite.
Candidatures reçues	Liste filtrable par offre et statut ; accès au profil/CV selon autorisation.
Détail candidature	Informations du candidat, CV, historique et changement de statut autorisé.
Entretiens	Planifier, modifier ou annuler un entretien et afficher le calendrier/listing.

5.4 Espace administrateur

- Dashboard global avec indicateurs utiles.
- Gestion des utilisateurs : recherche, consultation, activation/désactivation selon les règles serveur.
- Validation ou modération des entreprises et offres si le workflow le prévoit.
- Traitement des signalements et consultation des journaux autorisés.

6. Navigation et protection des routes

Routes publiques accessibles sans connexion ; routes privées accessibles uniquement avec une session valide ; routes par rôle accessibles selon le rôle renvoyé par l'API.

```
Public: /, /jobs, /jobs/:id, /login, /register
Candidate: /candidate/dashboard, /candidate/profile, /candidate/applications
Recruiter: /recruiter/dashboard, /recruiter/jobs, /recruiter/applications
Admin: /admin/dashboard, /admin/users, /admin/moderation
```

- Après connexion, rediriger vers le tableau de bord adapté au rôle.
- Si la session expire, afficher un message et rediriger vers la connexion.
- Afficher une page 403 pour accès non autorisé et une page 404 pour route inconnue.
- Ne jamais faire confiance à un rôle modifié uniquement dans le navigateur.

7. Design UI/UX et responsive

- Identité visuelle professionnelle, sobre et cohérente ; palette proposée : bleu marine, bleu électrique, blanc et gris clair.
- Typographie lisible, hiérarchie visuelle claire, boutons d'action distincts et formulaires aérés.
- Navigation adaptée : barre supérieure pour les pages publiques, sidebar pour les dashboards desktop, menu compact sur mobile.
- Breakpoints Tailwind : mobile d'abord, tablette puis desktop ; aucun débordement horizontal.
- Prévoir skeleton/loader, message d'erreur, confirmation de succès, empty state et état désactivé.
- Contraste suffisant, labels explicites, navigation clavier, focus visible et textes alternatifs pour les images.

8. Formulaires et validations frontend

Formulaire	Validations minimales
Inscription	Nom, prénom, email valide, mot de passe conforme, confirmation si prévue, consentement si requis.
Connexion	Email au format valide et mot de passe non vide.
Profil	Champs requis, longueurs maximales, format téléphone/URL si présents.
Offre	Titre, description, type de contrat, localisation et date limite cohérents.
Candidature	CV requis ou non selon la règle métier ; lettre facultative si autorisée.
Téléversement	Type MIME, extension et taille contrôlés côté interface, puis revalidés par le serveur.

Les validations frontend servent au confort utilisateur. Le backend reste la source de vérité et doit répéter toutes les validations.

9. Communication avec l'API

Créer un client HTTP centralisé avec URL de base configurable par variable d'environnement, gestion uniforme des erreurs et ajout du jeton selon la stratégie retenue.

- Services séparés : authService, jobService, profileService, applicationService, companyService, notificationService.
- Gérer les réponses 401, 403, 404, 409, 422 et 500 avec des messages adaptés.
- Prévoir pagination, filtres, tri et annulation des requêtes devenues inutiles.
- Ne pas coder en dur les données métier qui doivent venir du backend.

10. Sécurité côté frontend

- Utiliser HTTPS en production.
- Ne jamais afficher ni journaliser les mots de passe ou jetons.
- Définir avec le backend le stockage des jetons ; privilégier une stratégie sécurisée avec cookies HttpOnly si l'architecture le permet.
- Échapper/traiter les contenus affichés et éviter l'injection HTML non maîtrisée.

- Protéger les formulaires contre les doubles soumissions et afficher les expirations de session.
- Limiter les informations personnelles visibles selon le rôle et les autorisations API.

11. Performance, qualité et tests

- Découper les bundles et charger les espaces privés à la demande si nécessaire.
- Optimiser les images, éviter les rendus inutiles et afficher des états de chargement.
- Tests unitaires des composants et utilitaires ; tests des formulaires ; tests des routes protégées.
- Tests d'intégration des parcours : inscription, connexion, recherche, candidature, publication d'offre.
- Vérifier les principaux navigateurs modernes et les tailles mobile/tablette/desktop.
- Aucune erreur bloquante dans la console ; pas de lien ou bouton sans action.

12. Critères de validation et livrables

- Toutes les pages prévues sont accessibles selon le rôle.
- Les formulaires affichent les erreurs de manière claire et empêchent les soumissions invalides.
- Les appels API gèrent succès, erreurs, chargement et absence de données.
- L'interface est responsive et utilisable au clavier pour les parcours essentiels.
- Le dépôt contient un README frontend : installation, scripts, variables d'environnement, architecture et procédure de test.
- Livrables : code source React, composants, pages, services API, tests, documentation et captures des écrans principaux.

13. Ordre conseillé de réalisation

- Étape 1 : initialisation React, Tailwind, Router et structure des dossiers.
- Étape 2 : design system et composants communs.
- Étape 3 : pages publiques, inscription et connexion.
- Étape 4 : gestion de session et routes protégées.
- Étape 5 : espace candidat et parcours de candidature.
- Étape 6 : espace recruteur et gestion des offres.
- Étape 7 : espace administrateur et modération.
- Étape 8 : intégration API complète, tests responsive, accessibilité et documentation.